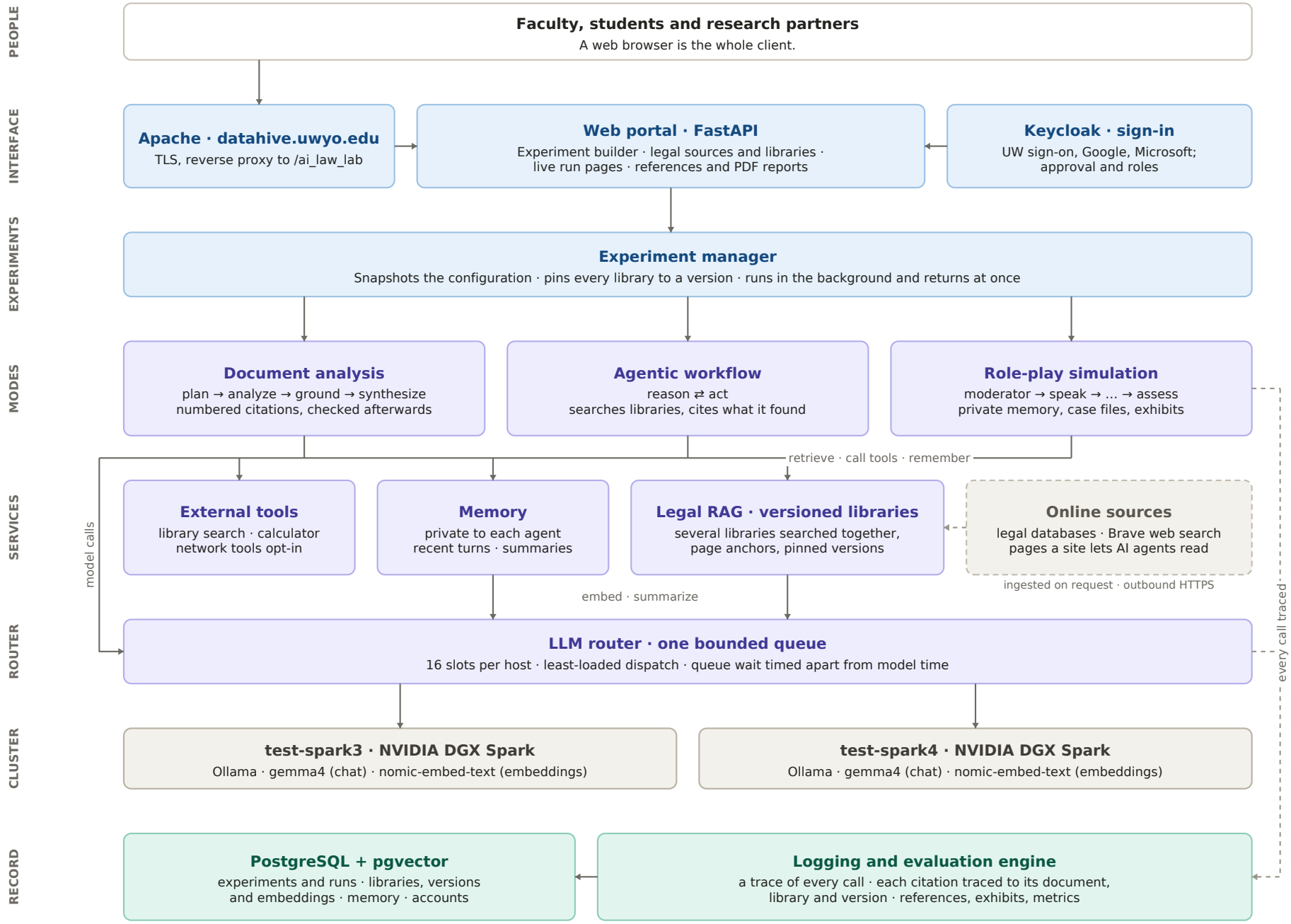
AI Law Lab: System Architecture

How a request becomes a traced, reproducible experiment run

Jian Gong · AI Law Lab, University of Wyoming · Architecture note, October 2026. Describes the software at commit dd9f6bd.



Reading the diagram

Rows are tiers, read top to bottom in the order a launched run passes through them. Solid arrows are that path. The two dashed lines are not part of it: on the right, the trace that everything above the cluster writes as it works; on the services row, the one outbound path, by which libraries are filled from public legal databases. Colour carries the meaning it has on the portal itself -- blue for the application, purple for anything that consumes model capacity, grey for hardware and outside services, green for what is kept.

- **Two hops, one process.** Apache terminates TLS and proxies to a uvicorn worker bound to loopback. There is exactly one application process; concurrency inside it is asyncio, not workers, which is why a single bounded queue can speak for the whole cluster.
- **Three modes, one set of services.** Document analysis, the agentic workflow and the role-play are three graphs over the same retrieval, tools, memory and tracer. They differ in their nodes, not in what they stand on.
- **Everything narrows at the router.** Every model call in every mode, and every embedding and summary the services need, passes through one admission point. That is what makes a timing number from one run comparable with another's.
- **Every run leaves a record that can be followed back.** The configuration it ran under, the version of every library it searched, every call it made, and for each citation the document, library and version it points to.

1 · Interface

The lab is served at `https://datahive.uwyo.edu/ai_law_lab/` from the same Apache vhost as the Open OnDemand portal. Three details in that configuration are load-bearing. The proxy passes the mount path through **unstripped**, because the application runs with `root_path=/ai_law_lab` and strips the prefix itself; proxying to a bare root would 404 its `/static` mount. `flushpackets=on` keeps the live trace stream from being buffered into uselessness. And `timeout 900` covers a model call that emits nothing while it is thinking -- with the default the browser would lose a role-play turn mid-generation. `X-Forwarded-Proto` is set on the location and uvicorn runs with `--proxy-headers`, so the application never emits `http://` URLs on an HSTS-pinned site.

Sign-in is a self-hosted Keycloak, served at `/sso/` (`/auth` belongs to another application on the same host). People sign in with UW sign-on, Google, Microsoft or a Keycloak account of their own. The portal runs OpenID Connect's authorization-code flow with PKCE itself (`web/auth.py`), with one unusual detail: this server cannot reach its own public address, so browsers are sent to Keycloak's public URL while the code exchange and signing keys go over loopback. Keycloak proves who someone is; the portal decides what they may do (`accounts.py`). A verified `uwyo.edu` address is approved as a researcher on first sign-in; anyone else (a Google or Microsoft account outside the university, say) fills in a short access request (name, organization, what they want to use the lab for, and viewer or researcher access), and an administrator approves it with a role or declines it with a note the person sees. A viewer reads and exports, a researcher also builds experiments, runs them and curates libraries, and an administrator also manages people and deletes things for good. Each request re-reads the person from the database, so disabling someone takes effect on their next click.

2 · Web portal

The interface is server-rendered Jinja2 with small amounts of JavaScript, not a single-page application. `views.py` turns stored settings into plain language, because the people configuring

experiments here are lawyers: an experiment page says how long a role-play is likely to take and shows each person in the cast as a card, with their case files.

Legal Sources is where libraries are built: search the online databases and ingest what is wanted, upload PDFs and text files, or add web pages by address and check them for changes later. Each library page lists its versions, what changed in each, the runs that searched each one, and a citation list for any version. **Run pages** show the answer or the role-play transcript with every citation marker linked to its document; the passages the model was given and which it cited; a References section listing the cited documents with the library version each came from and every place it was cited; for a role-play, the exhibits each side disclosed; and the metrics and the full trace. A run exports as a PDF report, and "run again with these versions" repeats it against the libraries exactly as it searched them. While a run is in progress its trace reaches the browser over server-sent events, polled from `run_events` once a second.

3 · Experiment manager

`experiments.py` is the seam between the portal and the graphs. Launching copies the experiment's configuration into `runs.config_snapshot` before any work starts, so editing an experiment afterwards cannot rewrite the conditions a past result was produced under. Libraries are versioned the same way: each library a run will search -- the experiment's own, or those chosen at launch, and in a role-play every agent's case files -- is pinned to a recorded version and written to `run_libraries`, so the result can be traced to, and re-run against, exactly the documents it searched.

Execution happens in a background task and the run id comes back immediately, which is what lets a browser watch an hour-long scene without holding a request open. A failure is recorded as a status with its trace intact, not raised: a failed experiment is a result. A role-play's cast is checked before the first model call, so a duplicate agent id fails at once rather than halfway through a scene.

4 · Experiment modes

Each mode is a LangGraph state graph over a typed state dictionary. Live services -- the router, tracer and the run's pinned libraries -- travel in `config.configurable` rather than in state, because state is checkpointed and connections are not serializable.

Mode	Nodes	Sources and citations
`document_analysis`	plan → analyze → ground → synthesize	Sub-questions are answered from the document concurrently; the findings are grounded in one search of all the run's libraries, and the answer cites the passages as [1], [2].
`agentic_workflow`	reason ⇄ act	The agent searches the libraries as a tool, all at once or one at a time. A passage keeps one number across all its searches, so a [3] in its answer names one passage. Ends when the model stops asking for tools.
`roleplay`	moderator → speak → moderator … → assess	Each turn the speaker sees passages from its own case files and the shared libraries as [S1], [S2]. Citing one discloses it as an exhibit, [E1], that everyone may cite afterwards. Ends when the moderator closes the scene or the turn limit is reached, never before everyone has spoken twice.

The role-play's rules -- who speaks next, when an impasse warrants intervention, when a scene is over -- and the studies behind them are in `graphs/roleplay_policy.py` and the companion note on role-play moderation.

5 · Shared services

- **Legal RAG** (`rag.py`). A library is a named set of documents, filled from uploads, the online databases (`sources.py`) or web pages (`web_links.py`). A document is chunked at 2400 characters with 300 of overlap, on paragraph boundaries, embedded, and kept with page anchors so a citation points where a person can check it. Documents are never edited in place: removing one sets `removed_at`, and re-reading a changed page adds a new row, so every recorded version of a library can still be searched exactly as it was. A run's libraries are searched together -- the top six passages above cosine 0.35, ranked against each other whichever library they come from, with text held in two libraries returned once -- and each passage is labelled with its library in the prompt.
- **Citations** (`grounding.py`, `citations.py`). Every citation marker a model writes becomes a record holding the passage, document, library and version it names, and where it appeared ("Answer", "Turn 7 · Dana Reyes"). A marker that names no passage the model was given is recorded as unsupported rather than dropped -- catching fabrication is the point. Each library also produces a citation list, one reference per document from what its source recorded, which is the form a run's References take.
- **Tools** (`tools.py`) is the registry the agentic loop is given: library search and a safe arithmetic evaluator. `allow_network` gates network-touching tools, and none are installed, so that switch currently changes nothing.
- **Memory** (`memory.py`) keeps a verbatim buffer inside a token budget and folds the overflow into model-written summaries that are embedded and recalled by similarity. Its scope is `(run_id, agent_id)`: in a negotiation, a shared buffer would leak one side's private reasoning into the other's context. Case files are private the same way: an agent's own libraries are searched only on its turns, and another agent sees a passage from one only after it is cited.
- **Tracer** (`tracing.py`) writes one sequenced row per model call, tool call, retrieval and node transition. Queue wait and model time go in separate columns because the benchmark showed them diverging threefold under load; reported together, a run that was merely waiting looks

like a run that was slow.

6 · Inference and the queue

Two DGX Spark units serve `gemma4:latest` through Ollama, reachable by Tailscale name only. The router in front of them is the piece of this architecture most worth explaining, because the obvious design is wrong. Benchmarking both hosts on 2026-07-23 found that fifty concurrent requests all complete -- nothing errors -- while aggregate throughput stops improving at about sixteen:

Concurrency	Aggregate tok/s	p50 latency
1	18	0.66 s
16	160	1.87 s
32	131	3.29 s
50	160	5.22 s

Past sixteen, the extra requests are queueing inside Ollama, not running. Round-robin dispatch would push fifty requests at a host and then report the resulting latency as though it were model speed.

So the router keeps a single global semaphore sized to the healthy hosts' combined capacity -- sixteen slots each, thirty-two in total -- and queues everything else itself. Waiting there is the backpressure, and the time spent is what gets recorded as `queue_wait_ms`. Admitted work goes to the least-loaded healthy host, which self-balances better than round-robin when one agent is summarizing a two-hundred-page contract and another is emitting a one-line reply. Hosts are probed every thirty seconds, parked after three consecutive failures, and the global capacity is rebuilt when the healthy count changes.

One model property shapes the tiers above: gemma4 reasons before answering by default. That reasoning is exactly what a research setting wants, and it is captured to its own column -- but it costs roughly an order of magnitude more output tokens on short answers. Nodes doing mechanical work (turn-taking, summarizing, planning) therefore pass `think=False`; nodes doing substantive legal reasoning pass `think=True`.

7 · Record

One PostgreSQL 17 database with pgvector, in a container on loopback, holds everything a run produces and everything it searched. It is reached through an async pool rather than a single connection, since about thirty-two agents can be in flight at once. Vectors are 768-dimensional and indexed with HNSW, chosen over IVFFlat because libraries grow incrementally and HNSW needs no retraining.

Tables	What they hold
`experiments, runs`	Definitions, and per-run snapshots of the configuration each result was produced under.
`run_libraries`	Each library a run searched, and the version it searched.
`run_events, citations`	The append-only trace, and every citation marker with its passage, document, library, version and a grounded / unsupported verdict.
`documents, chunks, corpus_versions`	The libraries: text, page anchors and embeddings, and the exact set of documents each version held.
`memory_messages, memory_long_term`	Short-term buffers and summarized long-term memory, per (run, agent).
`users, audit_log`	Who may use the lab and in what role, their access requests, and a log of requests, approvals, role changes and permanent deletions.

Everything a run produces is addressable by `run_id` and cascades from the run, so a result can be audited or removed as a unit. Libraries are kept apart from runs: deleting a run never touches what it searched.

8 · Outbound traffic

Model inference runs on the two Sparks; no prompt or document is sent to a commercial model API. The server makes outbound requests in one case, drawn dashed on the services row: fetching public material -- searches and documents from CourtListener, the Federal Register, the eCFR, govinfo and SEC EDGAR, open-web searches (Brave) and the pages a research agent reads, a web page added to a library or crawled into one, or a page or PDF a cast is drafted from. These carry a search term or an address, never an uploaded document.

That path is guarded, because this server can reach the Sparks, Postgres and the university network. Only `http/https` links whose host resolves exclusively to public addresses are fetched, and every redirect is re-checked the same way. Real people and private organizations in a source are renamed before a cast is drafted from it, and the same substitution is applied to the model's output: a draft invents bottom lines and confidential facts, and attaching those to a real company is the failure worth engineering against.

A page that a research agent or a crawl reads on the lab's own initiative is read only if its site lets AI agents read it (`site_rules.py`): robots.txt for the lab's own name and for well-known AI agents, a `noai` marking, and text-and-data-mining reservations (W3C TDMRep). Pages turned away are listed on the run page. The companion note Outside Data Access and Agent Web Reading in the AI Law Lab sets out each service, when a run goes online, and these checks in full.

Where each box lives

Box on the diagram	Source
Apache	/etc/apache2/sites-available/ood-portal.conf
Web portal	web/app.py, web/views.py, web/templates/, web/report_pdf.py, agent_spec.py, cast_assistant.py, source_material.py
Keycloak sign-in	web/auth.py, accounts.py, scripts/keycloak_setup.py, docs/keycloak.md
Experiment manager	experiments.py
Modes	graphs/document_analysis.py, graphs/agentic_workflow.py, graphs/roleplay.py, graphs/roleplay_policy.py, graphs/state.py
Legal RAG	rag.py, sources.py, web_links.py, crawler.py
Online sources	network_tools.py, site_rules.py, source_material.py
Tools, memory	tools.py, memory.py
LLM router	router.py
Logging and evaluation	tracing.py, grounding.py, citations.py
PostgreSQL	db.py, db/schema.sql, docker-compose.yml

Paths under `src/ailawlab/` unless absolute. Settings quoted throughout this note are defaults from `config.py`; every one is overridable by an `AILAWLAB_`-prefixed environment variable. The service runs as `ai-law-lab.service` under systemd.

Rebuilding this note

```
uv venv /tmp/docenv
uv pip install --python /tmp/docenv/bin/python reportlab
/tmp/docenv/bin/python docs/architecture/build.py
```

The diagram is laid out in `docs/architecture/build.py`; its rows and boxes are data at the top of that file. Re-run it after a change to the tiers, and re-run `scripts/bench_sparks.py` after any hardware or Ollama change, since the numbers in section 6 are what the router's slot count is set from.